Análise de boas práticas na implementação de APIs RESTful: uma revisão sistemática.

Alysson dos Anjos de Oliveira

[1]Departamento de Ciências Exatas e da Terra, Campus I
Universidade do Estado da Bahia (UNEB)
Salvador, Bahia, Brasil.

alyssonoliveira456@gmail.com

Resumo. *A motivação deste trabalho surge em decorrência do crescente número de grupos de pesquisa no Brasil, onde existe escassez para consumo de dados. A arquitetura Representational State Transfer (REST) tem ganhado reconhecimento no mundo Web para construções de Application Programming Interfaces(API) e hoje é um dos padrões de arquitetura mais utilizados no mundo. Esta revisão sistemática busca entender e investigar os principais antipadrões da arquitetura REST, falhas de segurança, problemas de semântica e quais as maneiras corretas de se construir uma API REST. A metodologia envolveu a criação de questões de pesquisa, formulação da busca, coleta de estudos, análise, avaliação de qualidade e extração de dados, que demonstrou diversos problemas de implementação REST. Esta revisão concluiu que, apesar de amplamente conhecida e utilizada, a implementação de APIs REST não é feita de forma adequada pelos desenvolvedores, a correta aplicação arquitetura introduz mais complexidade de desenvolvimento, porém também é responsável por trazer benefícios como segurança, escalabilidade e flexibilidade para sua construção.*

Palavras-chave: *RESTful APIs; Web APIs; Padrões de design; Engenharia de Software; Revisão sistemática da literatura.*

Abstract. *The motivation for this work arises from the growing number of research groups in Brazil, where a scarcity of data-providing APIs is observed. The Representational State Transfer (REST) architecture has gained significant prominence in the Web environment for the construction of APIs and is currently one of the most widely used architectural patterns in the world. This systematic review investigates the primary REST architecture antipatterns, security vulnerabilities, and semantic issues, while identifying the correct practices for building REST APIs. The methodology involved the creation of research questions, search formulation, study collection, analysis, quality assessment, and data extraction, which revealed several REST implementation problems. This review concluded that, although widely known and utilized, the implementation of REST APIs is often not performed adequately by developers. While the correct application of this architecture introduces greater development complexity, it is also responsible for providing benefits such as security, scalability, and flexibility to the system's construction.*

Keywords: *RESTful APIs; Web APIs; Design patterns; Software engineering; Systematic literature review.*

1. Introdução

A quantidade de grupos de pesquisa no Brasil tem mostrado um crescimento constante, todos os dias novos grupos de pesquisa surgem, porém não existem muitos meios de se conseguir informações a respeito dos grupos. O Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) construiu o Diretório de Grupos de Pesquisa (DGP), porém o sistema deles não provê uma API de fácil acesso para pessoas fora de instituições acadêmicas. Por conta disso surge a necessidade de se ter métodos alternativos de consumir esses dados por meio de uma API pública e de fácil acesso.

Para viabilizar esse acesso de forma eficiente, as APIs Web Application Programming Interfaces (API)s Web consolidaram-se como o serviço padrão para a interoperabilidade na internet. Historicamente, esses serviços baseavam-se em protocolos como Remote Procedure Call (RPC) ou Simple Object Access Protocol (SOAP) [Kumar et al. 2023]. A partir de 2000, com a proposição de Roy Fielding, a arquitetura Representational State Transfer (REST) emergiu como o paradigma predominante, sendo adotada por gigantes como Google e Facebook em processos de modernização de sistemas legados [Palma et al. 2022].

Contudo, a forte presença do modelo REST não garantiu a correção de sua aplicação. Evidências empíricas demonstram que uma parcela ínfima das API públicas cumpre integralmente os princípios arquiteturais documentados, apresentando falhas persistentes de semântica e segurança [Neumann et al. 2021]. A motivação desta pesquisa surge, portanto, da necessidade de investigar essas lacunas, unindo a prática de desenvolvimento ao rigor científico para identificar como evitar antipadrões e implementar estratégias de design seguro.

Esta pesquisa tem como objetivo identificar, através da realização de uma Revisão sistemática da literatura (RSL), práticas arquiteturais, mecanismos de segurança e antipadrões recorrentes na implementação de APIs REST, de modo a fundamentar o desenvolvimento de uma API pública para acesso a dados de grupos de pesquisa do CNPq com maior segurança, manutenibilidade e escalabilidade.

2. Relato da Revisão de Literatura

Foi definida uma RSL voltada a investigar como APIs REST são implementadas, considerando aspectos de semântica HTTP, segurança, modularização e reuso. Utilizou-se de strings de busca, critérios de inclusão e exclusão, definição de PICOC ,questões de pesquisa, análise qualitativa e quantitativa dos resultados, para permitir a coleta de artigos que possam fundamentar esta pesquisa.

2.1. Definição do PICOC

Para garantir que a estratégia de busca fosse ríspida e alinhada aos objetivos do trabalho, utilizou-se a estrutura PICOC (Population, Intervention, Comparison, Outcome, Context). Esta metodologia auxilia na extração de termos centrais para a formulação das *search strings*:

- **Population (População):** APIs baseadas no estilo arquitetural REST e serviços Web;
- **Intervention (Intervenção):** Implementação de APIs REST, arquitetura modular e aplicação de boas práticas (*Good Practices*);

- **Comparison (Comparação):** Uso de padrões de projeto (*Design Patterns*) e níveis de reuso de software;
- **Outcome (Resultado):** Indicadores de disponibilidade (*Availability*), segurança, eficiência de *cache* e escalabilidade;
- **Context (Contexto):** Disponibilização de dados para grupos de pesquisa e integração com o Diretório de Grupos de Pesquisa (DGP/CNPq).

2.2. Critérios de Inclusão e Exclusão

Para que essas perguntas pudessem ser respondidas foram usados critérios de inclusão e exclusão rigorosos, assim como uma string de busca específica para gerar resultados coerentes com o objetivo da pesquisa. Os critérios de exclusão(CE) utilizados foram:

- Estivessem fora do escopo principal do trabalho;
- Não focassem em APIs REST;
- Não abordassem organização arquitetural, modularização, manutenibilidade ou práticas relevantes para a implementação de APIs RESTful;
- Fossem anteriores ao ano de 2020;
- Não estavam escritos em inglês.

Os critérios de inclusão(CI) utilizados foram:

- Estudos que descrevem APIs REST;
- Estudos que descrevem arquiteturas de código;
- Estudos que abordam reuso de software ou componentes reutilizáveis;
- Estudos que discutem preocupações de segurança.

2.3. Questões de Pesquisa

Esta revisão sistemática busca responder a quatro questões fundamentais, que tem capacidade de fundamentar esse estudo, são elas:

- Quais mecanismos são relatados para mitigação de DDoS, bots e abuso de requisições em APIs RESTful?
- Quais os meios de autenticação e autorização mais utilizados;
- Quais as práticas recomendadas e tipos de arquitetura para a construção de APIs REST;
- Quais os antipadrões detectados nessa arquitetura.

2.4. Fontes de Dados e Estratégias de Busca

A coleta de dados foi realizada nas seguintes bases digitais

- **IEEE Xplore**: A pesquisa no IEEE <https://ieeexplore.ieee.org/Xplore/home.jsp> foi feita utilizando o mecanismo padrão de busca da plataforma, que faz a busca pelo *metadata* dos artigos;
- **ACM Library**: A pesquisa no ACM <https://dl.acm.org/> foi feita utilizando o mecanismo de busca *Advanced Search*, filtrando os artigos apenas pelo *abstract*.

Devido à amplitude dos temas, a *search string* foi dividida em dois eixos principais para garantir a sensibilidade da busca:

1. **Arquitetura e Reuso:** `("REST API"OR "RESTful") AND ("reusability"OR "software reusability"OR "design patterns"OR "modular design") NOT ("llm"OR "iot") ;`
2. **Segurança e Performance:** `("REST API"OR "RESTful") AND ("security"OR "authentication"OR "authorization") AND ("cache"OR "caching"OR "scalability") NOT ("iot"OR "llm").`

2.5. Avaliação de Qualidade

Após a filtragem inicial por título e resumo, os estudos candidatos foram submetidos a um checklist de qualidade com as seguintes perguntas:

- O trabalho propõe ou explica uma estrutura interna modular ou padrões de projeto para melhorar a manutenibilidade do código e o reuso?;
- O trabalho discute estratégias de armazenamento e cache?;
- O estudo descreve o ambiente experimental com nível de detalhe suficiente para permitir reprodução?;
- O trabalho segue ou descreve APIs RESTful adequadamente, incluindo uso correto de códigos de status, verbos HTTP e HATEOAS?;
- O trabalho utiliza TDD, BDD ou algum tipo de ferramenta para inspecionar vulnerabilidades ou robustez?;
- O trabalho aborda mecanismos de segurança em APIs, incluindo autenticação/autorização (JWT, OAuth2 etc.) ou proteção contra ameaças ativas, como DoS, rate limiting e mitigação de bots?

Essas perguntas foram escolhidas por buscarem exatamente o que essa RSL se propõe a investigar, sendo perguntas cruciais para determinar quais artigos possuem informação suficientes para fundamentar o estudo. Todas as perguntas de qualidade possuem o mesmo peso nesse trabalho, e as mesmas opções de resposta sendo elas: sim, parcialmente e não, valendo 1, 0.5 e 0 pontos, respectivamente. Apenas estudos com pontuação superior a **3.5/6.0** foram selecionados para a extração final de dados. A nota de 3.5 foi escolhida para se ter um alto rigor na hora de seleção dos artigos, para conseguir apenas aqueles que realmente podem contribuir significativamente para a construção deste estudo.

2.6. Resultados Parciais

O modelo de pesquisa foi feito com base na metodologia do PRISMA, sendo referenciado pela imagem 1



Figura 1. Metodologia de pesquisa baseado no PRISMA

A busca inicial retornou 128 artigos (119 na IEEE e 9 na ACM) descritos na figura 2. Após a aplicação dos critérios de exclusão e inclusão 22 artigos foram aceitos.

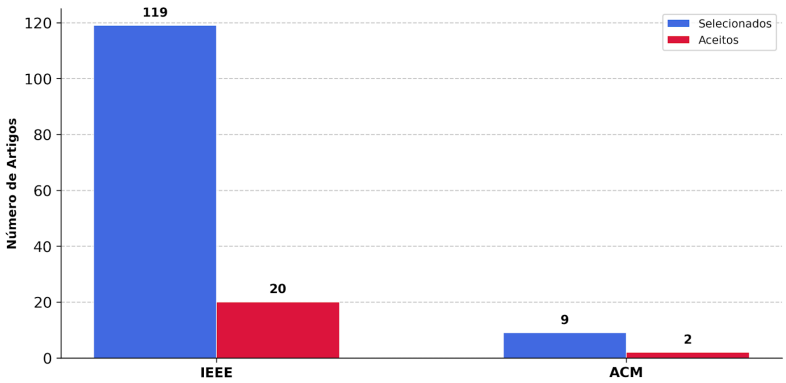


Figura 2. Número de artigos selecionados x Aceitos

Após a triagem inicial dos estudos uma segunda avaliação ríspida de qualidade foi realizada onde restaram 6 artigos de alta relevância que fundamentam esta revisão, descritos na tabela 1 , onde as linhas em negrito representam os que passaram na avaliação.

Tabela 1. Avaliação de Qualidade dos Artigos Candidatos

Título	Pontuação
Designing a Service-Oriented Platform on Digital Campus Using SCSE Framework	2.5
Conversation Disentanglement As-a-Service	3.0
Analysis of API Architecture: A Detailed Report	**4.0**
An Analysis of Public REST Web Service APIs	**4.0**
Vulnerability Assessment of Service Mesh-Based Microservices Having Workflow Dependencies	**4.5**
Pagination And Its Efficient Methods For RESTful Web Services	1.5
Modern Web Technologies in Financial Applications: A Performance-Driven Approach	3.0
An application about server communication: Using the Command pattern on Web API requests	3.0
REST API Access Control - an OPTIONS Based Approach	3.0
Detection of REST Patterns and Antipatterns: A Heuristics-Based Approach	**4.0**
Designing a Node.js Project Architecture for RESTful Microservices	3.5
Ensuring Data Coherence across Enterprise Systems and AI	2.5
A Mixed-Method Approach to Recommend Corrections and Correct REST Antipatterns	3.0
Securing APIs and Chaos Engineering	3.0
Comparative Study of Two Web Applications Developed Using MERN and MEAN Stacks	3.0
RESTful Web Services-Based Whistleblowing Systems Development for Higher Education	2.5
RESTful Web Service Architecture for Secure and Efficient Agricultural Data Integration in Jombang Regency	2.5
A Secure Digital Image Marketplace: Microservices and OWASP API Security Using Spring Boot	3.5
VOAPI2: a Vulnerability-Oriented Testing Framework for RESTful APIs	2.5
Improving the Security and Reliability of SDN Controller REST APIs Using JSON Web Token (JWT) with OpenID and auth2.0	3.0
Scalable Microservice Architecture for Secure E-Commerce Using REST, JWT, and Kafka	**4.0**
Vulnerabilities and Secure Design Strategies in RESTful Web APIs: A Systematic Literature Review	**4.5**

2.7. Análise Qualitativa dos Resultados

A análise dos 6 artigos restantes gerou conteúdo relevante para a pesquisa e suas questões, contribuindo para um melhor entendimento do que é uma boa arquitetura REST e como ela deve se comportar. Os principais artigos considerados nessa análise foram [Tamariz-Moreno et al. 2025] [Sivakumar K 2025] A forma como os estudos respondem as questões de pesquisa estão abaixo:

QP1: Quais mecanismos são relatados para mitigação de DDoS, bots e abuso de requisições em APIs RESTful?

É comum no desenvolvimento de APIs utilizar diversas camadas de defesa técnica para mitigar quaisquer tipos de ataques, dentre elas identificou-se:

- **Limitação de Taxa (Rate Limiting)** e Quotas: É a prática mais comum para evitar sobrecarga, limitando o número de chamadas permitidas por usuário ou aplicação em um período definido de tempo [Neumann et al. 2021] e [Tamariz-Moreno et al. 2025], ambos com nota 4.5;
- **API Gateway e Proxy Reverso**: Funciona como um escudo centralizado que aplica políticas de segurança, bloqueia requisições maliciosas e gerencia o tráfego [Tamariz-Moreno et al. 2025], todas as requisições vão precisar passar por esse escudo antes de chegar ao servidor principal [Prasad et al. 2025];
- **Detecção de Anomalias e Tráfego Não Humano**: O uso de CAPTCHAs, técnicas de detecção de interação não humana e sistemas de detecção de intrusão (IDS) que utilizam machine learning para identificar padrões de ataque em tempo real [Tamariz-Moreno et al. 2025];
- **Balanceamento de Carga e *Circuitbreaking***: Apesar de não ter sido muito comentando é uma técnica que pode mitigar as vulnerabilidades do sistema à ataques Distributed Denial of Service (DDoS) [Sivakumar K 2025].

QP2 - Quais os meios de autenticação e autorização mais utilizados

APIs REST foram feitas para funcionar sem manter o estado atual, por isso seus meios principais de autenticação e autorização utilizam métodos que garantem isso, sem perder a segurança do sistema. Apesar de nenhum artigo ter explicado explicitamente a diferença entre autenticação e autorização, eles fornecem explicações do que eles representam. O primeiro representa o processo de identificação do usuário,o momento em que ele se registra, faz login na aplicação [Prasad et al. 2025], já o segundo representa as permissões que o usuário possui ao se autenticar na aplicação, normalmente feito utilizando métodos como RBAC [Tamariz-Moreno et al. 2025]

- **OAuth**: Amplamente adotado como o padrão para autorização, sendo utilizado pela maioria das APIs públicas. Atualmente possui duas versões OAuth2.0 e OAuth1.0, onde a primeira oferece melhorias significativas em relação a primeira versão [Neumann et al. 2021];
- **JSON Web Tokens (JWT)**: Frequentemente utilizado em arquiteturas de microserviços por permitir a transmissão segura e compacta de informações de identidade sem necessidade de armazenamento de sessão no servidor [Prasad et al. 2025];
- PASETO e MFA: O PASETO sendo citado como uma alternativa mais segura ao JWT por oferecer criptografia nativa e evitar vulnerabilidades comuns de implementação , e o MFA como um método complementar para incrementar a segurança de outros mecanismos de defesa [Tamariz-Moreno et al. 2025].

QP3 - Quais as práticas recomendadas e tipos de arquitetura para a construção de APIs REST

Historicamente, o desenvolvimento de APIs fundamentava-se em arquiteturas monolíticas, nas quais uma única unidade lógica era responsável por processos distintos, como autenticação e persistência de dados. Embora comum, esse paradigma resulta em alto acoplamento e limita a escalabilidade, tornando o ciclo de implantação (deploy) complexo e arriscado[Prasad et al. 2025]. Em contrapartida, a arquitetura de microserviços propõe a decomposição do sistema em serviços independentes e fracamente acoplados, que executam funções específicas e comunicam-se via protocolos leves, garantindo maior resiliência e flexibilidade operacional[Sivakumar K 2025].

A construção de APIs em arquiteturas de microserviços segue o princípio de decomposição de domínios e independência entre serviços, onde os estudos analisados destacam:

- **Service-per-Domain**: cada funcionalidade (autenticação, catálogo e pedidos) é tratada como um serviço independente, com seu próprio banco de dados [Prasad et al. 2025];
- **Service Discovery (Eureka)**: as APIs se registram em um diretório dinâmico, permitindo que o sistema localize instâncias de serviços sem depender de IPs estáticos [Prasad et al. 2025];
- **Orquestração via API Gateway**: o gateway recebe a requisição do cliente, valida o token JWT e roteia a chamada para o microserviço alvo [Prasad et al. 2025, Tamariz-Moreno et al. 2025];
- **Mensageria assíncrona (Kafka)**: utilizada para distribuir dados e eventos entre serviços sem criar dependências de tempo de resposta, aumentando a tolerância a falhas [Prasad et al. 2025].

Esses elementos mostram que a resposta para a QP3 está associada a desacoplamento, descoberta dinâmica de serviços, mediação centralizada de acesso e comunicação assíncrona. A partir dessa base arquitetural, a QP4 discute quais erros e antipadrões comprometem esse tipo de construção.

QP4 - Quais as más praticas detectadas na construção de APIs REST

Más práticas ou antipadrões são o oposto dos padrões de design e podem gerar diversos problemas na aplicação, impactando diretamente desempenho, evolução e manutenção do sistema [Sivakumar K 2025]. Esses problemas são comuns nas fases iniciais do desenvolvimento e fazem com que manter ou evoluir as APIs se torne uma tarefa complexa [Palma et al. 2022]. Abaixo estão os principais antipadrões encontrados nos estudos:

- **RPC-style URIs**: Incluir verbos de operação na URL (ex.: /deleteUser) em vez de usar os métodos HTTP adequados em URIs de recursos [Neumann et al. 2021];
- **Mass Assignment**: Permitir que o cliente modifique campos sensíveis ou protegidos por falta de filtragem nas propriedades de entrada [Tamariz-Moreno et al. 2025];

- **Falta de Hypermedia as the Engine of Application State (HATEOAS)**:Embora seja um padrão um tanto controverso e ignorado pela maioria das aplicações pela complexidade e overheading que cria, o não fornecimento de links para recursos relacionados também é um antipadrão [Palma et al. 2022];
- **Implementação inadequada de CORS**: configurações permissivas de *Cross-Origin Resource Sharing* que podem causar efeitos colaterais indesejados e exposição de dados [Tamariz-Moreno et al. 2025];
- **Acoplamento em monólitos**: dependência excessiva entre componentes, o que impede a escalabilidade independente e motiva a migração para microserviços [Prasad et al. 2025];
- **Breaking Self-descriptiveness**: uso de cabeçalhos customizados ou protocolos não padronizados [Palma et al. 2022];
- **Ignoring Caching/MIME Types/Status Codes**: não utilizar as capacidades nativas do HTTP para cache, múltiplos formatos de arquivo ou códigos de erro semânticos [Palma et al. 2022];
- **Tunnelling through GET/POST**: uso incorreto de métodos HTTP para operações inadequadas [Palma et al. 2022].

Tabela 2. Síntese dos Resultados por Questão de Pesquisa

Questão de Pesquisa (QP)	Resumo dos Achados e Práticas Identificadas
Q1: Quais mecanismos são relatados para mitigação de DDoS, bots e abuso de requisições em APIs RESTful?	Uso de camadas defensivas como *API Gateway* para filtragem, *Rate Limiting* para controle de carga, detecção de anomalias com ML, e mecanismos de *Circuit Breaking* para resiliência.
QP2 - Quais os meios de autenticação e autorização mais utilizados	Predomínio de métodos *stateless*: OAuth 2.0 para autorização e JWT para transmissão compacta de identidade. PASETO e MFA surgem como tendências para elevar a segurança nativa e proteção de acesso.
QP3 - Quais as práticas recomendadas e tipos de arquitetura para a construção de APIs REST	Migração para Microserviços baseada em *Service-per-Domain*, descoberta dinâmica via Eureka, orquestração centralizada por Gateway e comunicação assíncrona com Kafka para reduzir acoplamento.
QP4 - Quais as más praticas detectadas na construção de APIs REST	Identificação de falhas como URIs em estilo RPC (verbos na URL), falta de HATEOAS, *Mass Assignment*, configurações de CORS permissivas e o não aproveitamento de recursos nativos do HTTP como cache e *status codes*.

3. Conclusões

A análise dos resultados trouxe resultados interessantes conseguiram responder todas as questões de pesquisa sugeridas. No decorrer da revisão sistemática identificou-se que REST é um padrão muito conhecido porém pouco aplicado da maneira correta,seguindo todos os seus padrões de arquitetura e boas práticas.

Os estudos demonstraram que as empresas tem migrado do padrão de monólitos para arquitetura baseada em microserviços, que garantem uma escalabili-

dade,flexibilidade superior. Entretanto é importante ressaltar que essa mudança de arquitetura exige a aplicações de padrões bem definidos para atingir seu melhor funcionamento como *API Gateway*, modularização, segurança e dentre outros, para que seja possível construir algo seguro e escalar

Além disso, a dominância de diversos antipadrões relatados nos artigos, demonstram que a simplicidade que a arquitetura REST permite, pode acabar induzindo desenvoldedores a não aplicar suas melhores práticas, ferindo aspectos como segurança, escalabilidade, flexibilidade.Observou-se que a ausência de estratégias como Rate Limiting e o uso inadequado de verbos HTTP não apenas ferem a arquitetura, mas expõem as aplicações a vulnerabilidades críticas de autenticação e ataques de negação de serviço.

Por fim, esta RSL forneceu a fundamentação teórica necessária para o desenvolvimento de uma API REST voltada a fornecer dados relacionados aos grupos de pesquisa do Brasil. Os padrões de design seguro e as estratégias de reuso identificadas serão os pilares para a construção de uma ferramenta que não apenas provê dados com alta disponibilidade, mas que serve como um exemplo de implementação RESTful.

Referências

Kumar, K., Jain, A. K., Tiwari, R. G., Jain, N., Gautam, V., and Trivedi, N. K. (2023). Analysis of api architecture: A detailed report. In *2023 IEEE 12th International Conference on Communication Systems and Network Technologies (CSNT)*, pages 880–884. IEEE.

Neumann, A., Laranjeiro, N., and Bernardino, J. (2021). An analysis of public rest web service apis. *IEEE Transactions on Services Computing*, 14(4):957–970.

Palma, F., Moha, N., Tremblay, G., and Guéhéneuc, Y.-G. (2022). Detection of rest patterns and antipatterns: A heuristics-based approach. In *Service-Oriented Computing*, page 230–244.

Prasad, S., C, S., and Srivani, S. (2025). Scalable microservice architecture for secure e-commerce using rest, jwt, and kafka. pages 1–6.

Sivakumar K, S. T. P. (2025). Vulnerability assessment of service mesh-based microservices having workflow dependencies. *2025 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*, pages 189–194.

Tamariz-Moreno, A., Carrera-Colorado, E., Ocharán-Hernández, J., Pérez-Arriaga, J. C., and Limón, X. (2025). Vulnerabilities and secure design strategies in restful web apis: A systematic literature review. pages 233–242.

4. Planilha-resumo de Resultados

Tabela 3. Planilha de Extração de Dados alinhada aos Critérios de Inclusão

Identificação	CI1	CI2	CI3	CI4	Descrição	Avaliação
Prasad et al. (2025). Scalable Microservice Architecture for Secure E-Commerce Using REST, JWT, and Kafka		X	X		Spring Boot, Gateway e Kafka. Foco em escalabilidade de e-commerce.	Padrões API Gateway e Service Discovery para microserviços.
Kumar et al. (2023). Analysis of API Architecture: A Detailed Report	X				Analisa performance de REST e COAP com compressão GZIP.	Foca em cache via ETAG e dados experimentais de latência.
Neumann et al. (2021). An Analysis of Public REST Web Service APIs	X			X	Revisão de 500 APIs públicas para avaliar princípios REST.	Analisa conformidade de verbos HTTP e uso de HATEOAS.
Sivakumar (2025). Vulnerability Assessment of Service Mesh-Based Microservices Having Workflow Dependencies	X		X	X	Estudo sobre Service Mesh (Istio) e resiliência de workflows.	Identifica antipadrões cíclicos e estratégias de Circuit Breaking.
Tamariz-Moreno (2025). Vulnerabilities and Secure Design Strategies in RESTful Web APIs	X	X		X	Mapeamento de vulnerabilidades e estratégias de mitigação.	Roadmap para design seguro usando a taxonomia STRIDE.
Palma et al. (2022). Detection of REST Patterns and Antipatterns: A Heuristics-Based Approach				X	Detecção automática de 13 antipadrões via algoritmos heurísticos.	Precisão de 89.4% na detecção de falhas de design REST.